

Project Report AS - Gabo

Baptiste FOUGERAY

June 11, 2025

Contents

1	Project Description	2
1.1	Motivation	2
1.2	Public Dataset	2
2	Tasks Completed	2
3	Deployment and Usage Guide	25
3.1	Docker Compose local deployment	25
3.2	Docker Compose deployment Access	25
3.3	Kubernetes deployment Access	26
3.4	Usage Guide	27
4	GitHub Repository and DockerHub Images	27
5	Conclusion	27

1 Project Description

1.1 Motivation

Since the high school, one card game is a favorite among our group of friends. This game is called Gabo, and no one knows really where it comes from, especially because there are many different variants. We enjoy playing this game for countless hours. In short term, the main purpose is to do several rounds and the first one reaching 100 points loses and ends the game. Each player gets a score on 100, the purpose is to be the one with the least points at the end of the game - if you want further information about Gabo, please check the rules in the web app. The idea came to create a platform where players could fill the games, allowing one to build statistics, historic, and, the most important, an elo system to truly know which one of us is the best.

The Gabo application allows users to track any other player through an online interface. Players can be registered in the system with unique profiles that allow them to participate in recorded matches. The application provides immediate updates on game statistics and player rankings maintaining an accurate and current competitive environment. Each player profile contains performance metrics such as elo, total score, average score or relative average position in a game, making it simple to track progress and compare statistics with other players. The system maintains a complete historical record of all matches.

1.2 Public Dataset

On the way to realize this application, I asked myself how to display the rules in several different languages, such as English, Euskera, Spanish, French and Breizhoneg, of the game efficiently. I realized that creating my own kaggle dataset with rules in the different languages (csv format) allows to fetch the rules without "hard-coding" it in the codebase allowing for a more dynamic update of rules. For more details on the dataset, visit the Gabo Rules Dataset on Kaggle.

2 Tasks Completed

- Reflection on the Project Structure

Firstly I had to think about the structure of the containers that I will have to use. The front-end application (APP container) serves as the user interface layer providing multiple views and interactions through a combination of HTML templates and JavaScript modules. A central MySQL database container (Data_Base) maintains all persistent data while a dedicated backup service container ensures data reliability with scheduled backups and ensure reload of the last backup (moreover it is a

practical way to get or push a backup into the docker compose deployment).

I decided to go for three Apis:

- For fetching the rules **API-RULES**
- For managing new players and display scoreboard **API-AP**
- For managing new games **API-AG**

The entire system is orchestrated using Kubernetes with specific deployment configurations for Google Kubernetes Engine. All services communicate through a network configuration managed by Nginx that handles load balancing and routing.

In some cases, the architecture in microservices might seem overkill, but I wanted to be sure that the web can respond quickly, avoiding long waits in endpoints, efficiently scale efficiently and ensure an easier way to maintain. Moreover it will allow to create new features as it is DevOps oriented.

Let's move deeper in each containers.

– Web Application Development - APP Container

The `app_back_end.py` script uses Flask to create a web application. This container is only for the front end, so you will only find HTML, CSS, and JavaScript files.

The front-end interface consists of 5 distinct HTML pages, each extending from a base template (`base.html`). The `accueil.html` page serves as the home page, welcoming users to the application and allowing them to navigate through all others templates.

The `ajouter_joueur.html` (`add_player.html`) page provides a form interface for player registration. You just have to enter an id and a name that will be send to API-AP. In case the id is already taken, as it's an unique primary key in the Player table of the Data base, the web app will ask you to retry with an other id.

The `ajouter_partie.html` (`add_game.html`) page implements a table that that can be fill by searching players by Name or Id. The conversion from names to ids (the database only searching by id for optimization purpose) will be perform in the API-AG.

The `rules.html` displays game regulations fetched from the Kaggle dataset through out the API-RULES allowing displaying rules in English, Castellano, Euskera, Français and Breizhoneg.

The `scoreboard.html` page presents player rankings and statistics. It's the only page where some work has been done on CSS style and a JavaScript display logic. You can search by name or ids depending

of the toggle button. A table with all player by ids and an other by name is build and put in cache for 10 minutes to avoid redundant load and reduce pressure on the APIS. In the Kubernetes deployment cache is set to less than 1 second, then you will be able to test and immediately see updates from adding a player or a game.

The JavaScript architecture is organized into two main categories: display and API interactions. The display modules (`*-display.js`) handle user interface dynamics such as form submissions, table updates, and dynamic content rendering, while the API modules manage asynchronous communications with the backend services.

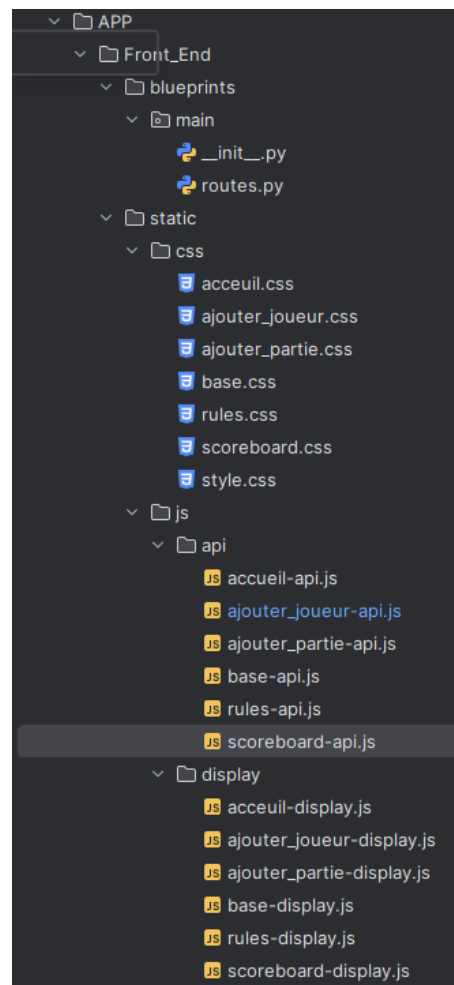


Figure 1: Static APP structure (Front End)

– Dockerfile-APP

This Dockerfile is crafted to build a container for a Python application served with Gunicorn. I decided to go for Gunicorn to get a more robust server for production than the native server from Flask that can be only use for debugging and testing. It starts from the base image `python:3.9-slim`.

The `WORKDIR /APP` command sets the working directory within the container to `/APP`, where the application code will reside.

Next, the command `COPY requirements.txt .` copies the `requirements.txt` file into the container. This file should list all necessary Python dependencies—including Gunicorn.

The `RUN pip install -no-cache-dir -r requirements.txt` instruction installs these dependencies without using pip's cache.

Then, `COPY . /APP` copies the entire codebase into the container's working directory (`/APP`).

Finally, the `CMD ["gunicorn", "-w", "1", "-threads", "4", "-k", "gthread", "-b", "0.0.0.0:8000", "BE_app:app"]` command starts Gunicorn to serve the application. It launches Gunicorn with one worker and one threads (using the `gthread` worker class), binds to port 8000 on all interfaces, and specifies the WSGI entry point as `BE_app:app`.

– API-RULES Script for Fetching and Serving Rules Datasets

This Python script sets up a Flask application that provides an API endpoint to fetch a game rules dataset in various languages using the Kaggle API. On startup, the script configures logging to output informational messages and errors. It defines several constants, such as the Kaggle dataset slug, download directory, and CSV filename paths for different languages (English, Spanish, French, Basque and Breton).

The core functionality revolves around two main components:

- * **Dataset Download and Extraction:** The `download_and_extract_dataset` function ensures the dataset is locally available. It creates the download directory even if it does exist (no need to deploy again if changes in the dataset are made), authenticates with the Kaggle API, and downloads all dataset files (unpacking them if necessary). This design enables dynamic updates of the game rules.
- * **Data Retrieval Endpoint:** The route `/api-rules/fetch_rules`, implemented with the `@app.route` decorator, handles GET requests. It accepts a `lang` parameter to determine which language-specific CSV file to use. The script logs the chosen language and file path, calls the download function, and then verifies the file's existence. If the CSV file is found, it reads the file into a Pandas

DataFrame (employing the Python engine to handle any potential formatting issues) and converts the data into a dictionary, which is then returned as a JSON payload. Any errors during the file reading or processing steps are logged and result in an error JSON response.

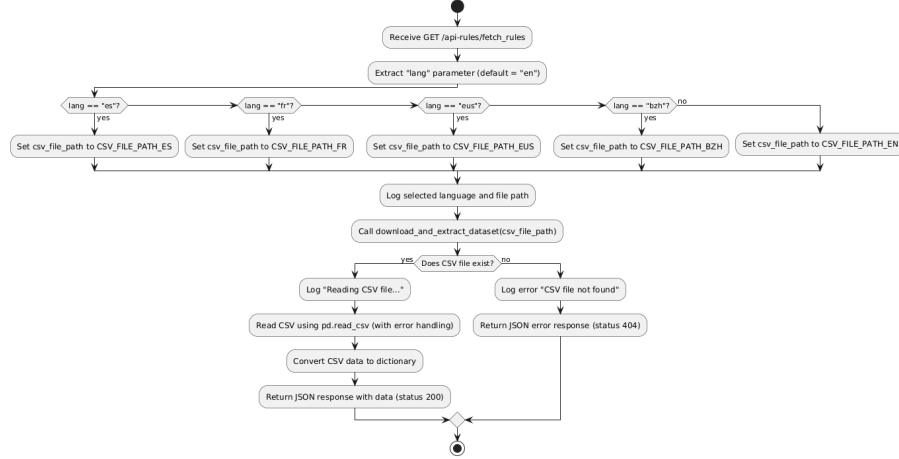


Figure 2: Activity diagram API-RULES

– API-RULES Dockerfile

This Dockerfile is barely exactly the same as the APP’s one.

The application is configured to run on host 0.0.0.0 and port 8030 with debugging enabled.

– API-AG Handling adding players and displaying info in Scoreboard page

This Python script configures a Flask API that handles player data management. It uses several utility functions to interact with a MySQL database and to notify external services when player data is updated. The script starts by importing the required modules, including Flask, logging, requests, and the custom utilities for database connection and error handling.

Key functionalities of the API include:

- * **Player Insertion:** The `send_to_database_j` function handles inserting player data into the `Joueurs` table. It uses a prepared SQL statement for security, logs the operation, commits the transaction, and calls `notify_service` to inform another service upon success. Errors during the process trigger a rollback and lead to a `DatabaseError`.

* **Mapping Update:** The `patch_mapping_index` function sends a PATCH request to the API-AG (defined by the `API_AG_URL` constant) to update mapping index data for the player. It logs the request and handles any network issues.

* **API Endpoints:**

- `/ready` marks the API as ready to accept requests. It sets a global flag, logs readiness, attempts a start-up process (including connecting to the database), and returns a confirmation message. It also builds the index map of players' name with their ids allowing to search in DB only by id which is a quick lookup. It is performed two lookups instead of one with a WHERE statement but it's still make it to theoretically quicker (Cpython dictionary looks in a C function underneath).

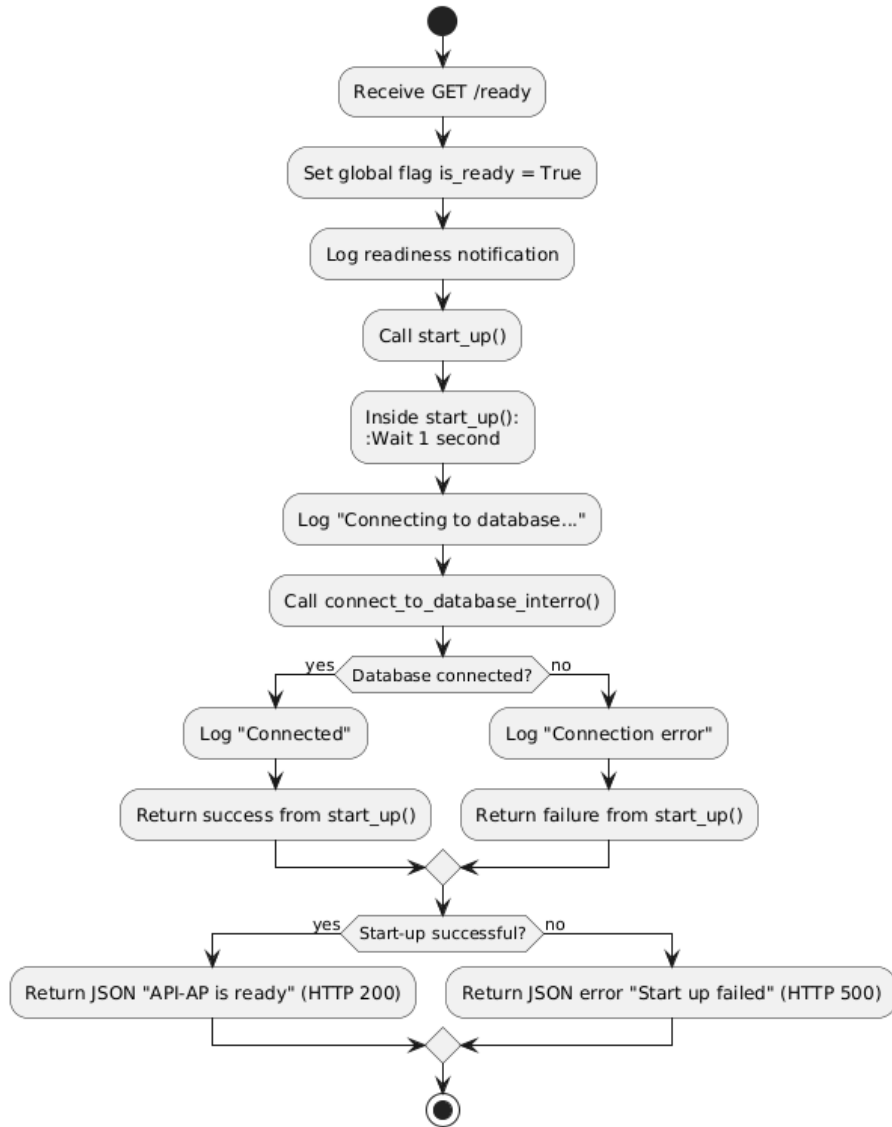


Figure 3: Activity diagram Startup API-AP

- `/api-ap/save_player` accepts a JSON POST request with `playerId` and `playername`. It validates input, saves the player to the database via `send_to_database_j`, and updates its own mapping index and the one of API-AG via `patch_mapping_index`. It returns success or error messages based on the outcome.

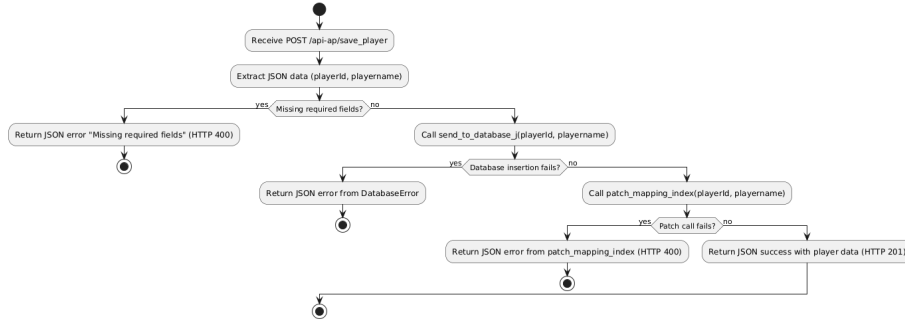


Figure 4: Activity diagram Save Player API-AP

- `/api-ap/get_player_info` retrieves information for a specific player based on a query parameter. It fetches data from the database and returns it as JSON, or a 404 error if the player is not found.

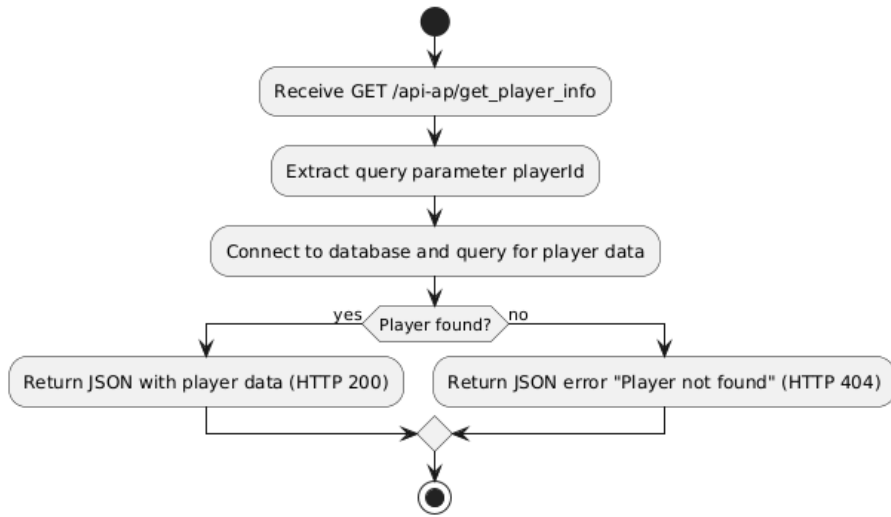


Figure 5: Activity diagram Get Player Info API-AP

- `/api-ap/get_all_players` calls `get_all_players` to retrieve and return all player records, sorted by their ranking (elo) in descending order.

– API-AP Dockerfile

This Dockerfile is barely exactly the same as the APP's one.

The application is configured to run on host 0.0.0.0 and port 8010 with debugging enabled.

Overall, this API container script integrates database and network operations to manage player profiles.

– API-AG Container Script for Game Session

This API script is built using Flask and integrates multiple functionalities to manage game sessions. It interacts with a MySQL database to store game results. I decided to go for a threaded architecture to test how it could work but in fact it is not already test but there is theoretically thread-safe operations via a global lock.

The core components of the API include:

* **RESTful API Endpoints:**

· **/api-ag/save_table**

Receives a POST request with a JSON payload containing a game result table. It processes the table by sorting it, calculating Elo variations, and then enqueues each game session. The queue is emptied subsequently to persist all game sessions into the MySQL database.

· **Data Sorting:**

The `sort_table` function implements an insertion sort on the table of player IDs, scores, and Elo ratings. This enables the API to compute and assign rankings for players without transferring additional ranking data from the front end.

· **Thread Safety:**

A global threading lock is used to guard critical sections, ensuring that modifications of shared resources (such as the global game ID and the queue operations) are handled safely in a concurrent environment.

* **Queue and Game Session Management:**

A `Queue` class is defined to manage a queue of game sessions, represented by the `Game` class. Each game session stores details such as the game ID, number of players, player ID, score, rank, and the Elo rating variation. The `insertion` function processes a table of game results by computing Elo adjustments (using `get_var_elo` and `calculate_elo`, as it is one against all elo variation is calculated for each pairs of player in the game using the probability of win of the winner multiply by the score difference. It seems a sound approach because there is variation that do not exceed 20 for a game (player are initialized at 500 while creating their profile) a test script, runnable with default python

interpreter, is available in the container if you want to see how elo behave depending of number of players in game, elo differences or score differences. It also creates game session instances, and inserts them into the queue. Afterwards, the `empty_queue` method processes these sessions, sending each game's data to the database using the `send_to_database` function.

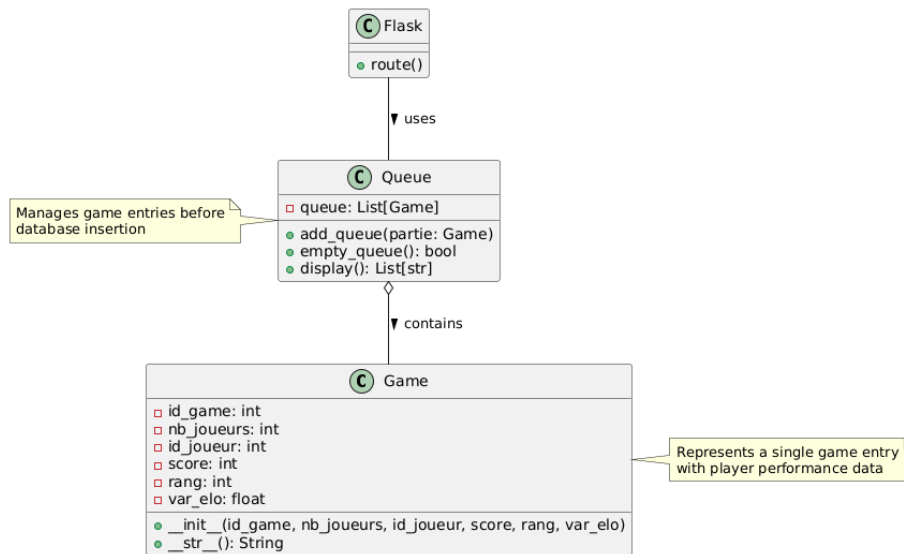


Figure 6: Class Diagram for Queue to DB

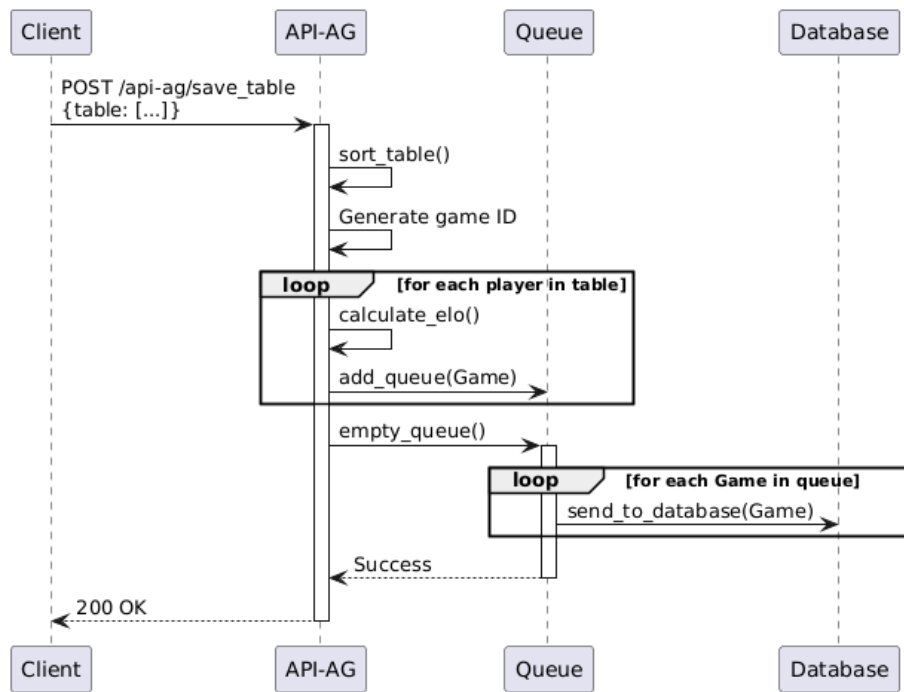


Figure 7: Sequence diagram Save Table

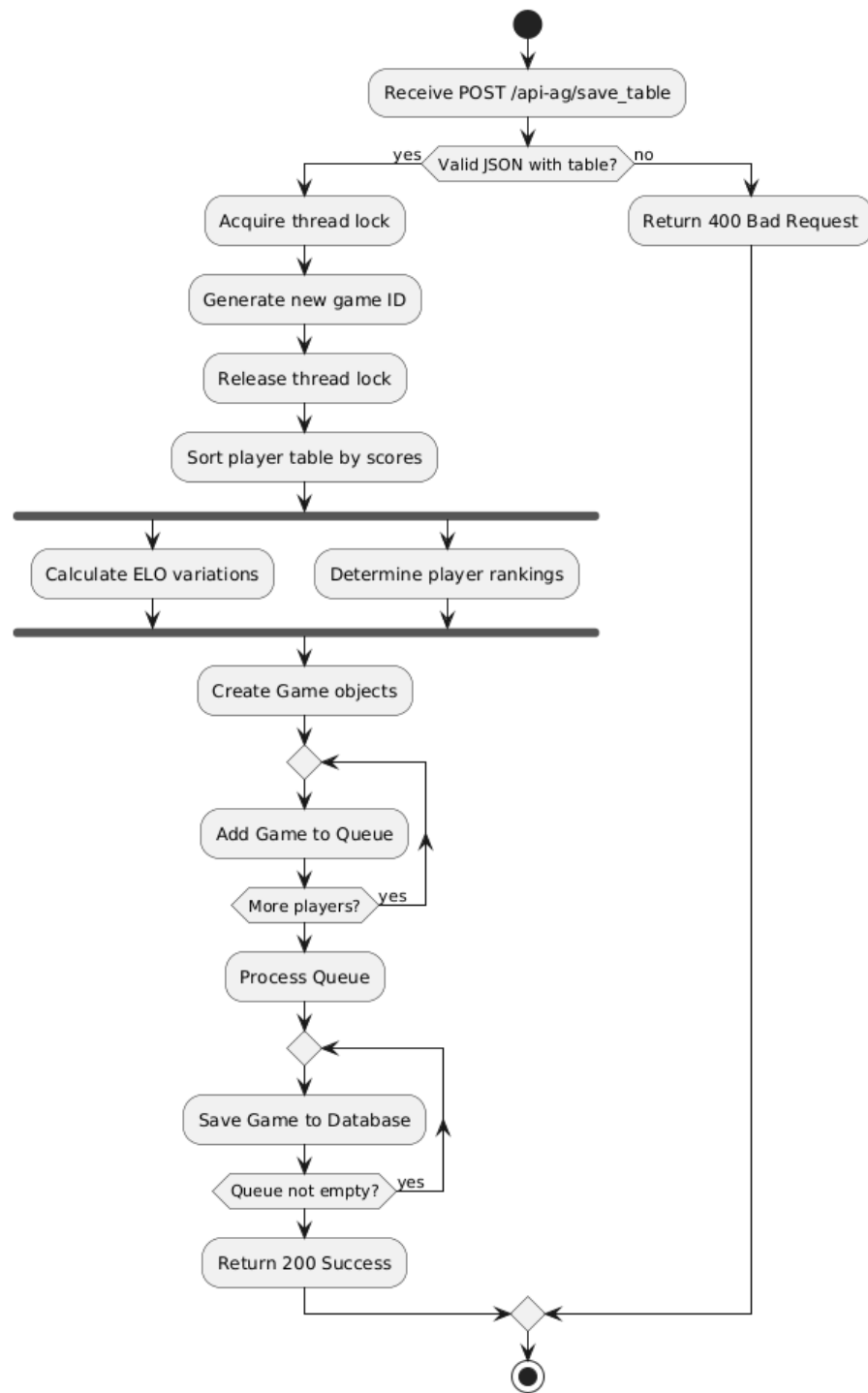


Figure 8: Activity diagram Save Table

- * `/api-ag/get_player_info`
Barely the same function as in API-AP.
- * `/patch_mapping_index`
A PATCH endpoint that updates the in-memory index mapping player names to their IDs, ensuring fast lookups without redundant database queries.
- * `/reload_indexbyname`
Comming from Backup each to two hours container and reloads the complete player name-to-ID mapping from the database into memory to ensure that the index is up to date.
- * `/ready`
Really similar to the API-AP except the fetch from DB retrieving the max id game. Marks the application as ready and initializes the startup process. During startup, the API attempts a database connection, creates the player index, and sets a global game ID counter based on the maximum game ID in the database.

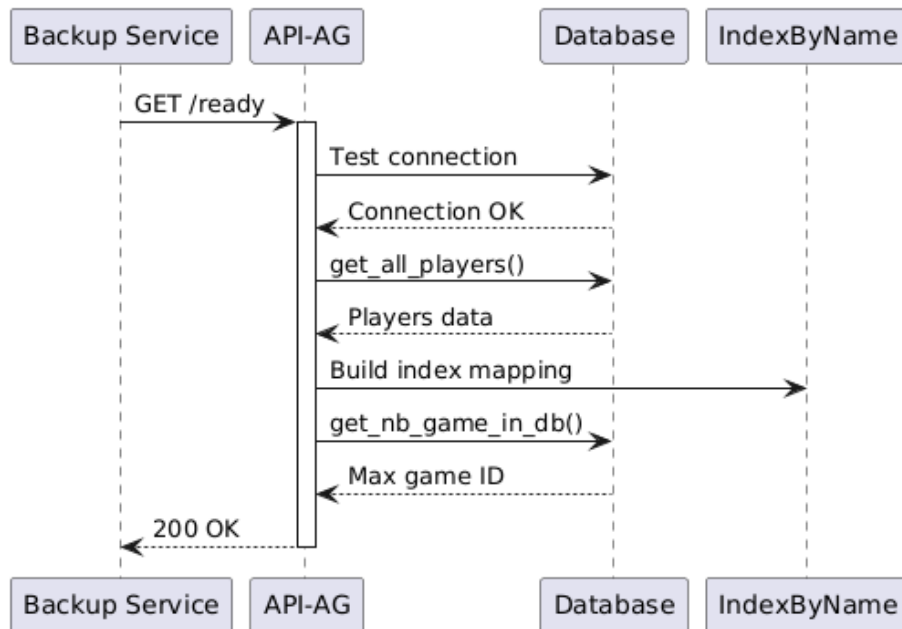


Figure 9: Sequence diagram Startup API-AG

– API-AG Dockerfile

This Dockerfile is barely exactly the same as the APP's one.

The application is configured to run on host 0.0.0.0 and port 8020 with debugging enabled.

– Backup Container Script for Database Backup, Restoration, and Service Startup Notification

This standalone service is responsible for managing database backups and restorations, while also coordinating readiness startup notifications with other components of the system. It is built using Python and leverages several external utilities to interact with MySQL as well as to trigger service notifications.

The API code is organized into the following major sections:

* **Database Readiness and Backup Restoration:**

- **wait_for_database:** Continuously pings the database using `mysqladmin` until a successful response is received, pausing for a fixed interval between attempts.
- **restore_latest_backup:** Scans the specified `backup_directory` for the most recent SQL backup file.

* **Backup Generation and Maintenance:**

- **generate_backup:** Uses `mysqldump` to create a database backup file. The file is timestamped and stored in a designated backup directory, which is created if it does not exist.
- If a previous backup exists (**last_backup**), the function attempts to remove it, thereby ensuring that only the latest backup is retained.

* **Service Notification:**

- After the database is confirmed ready and the latest backup is restored, the script invokes an external notification function (**notify_service**) to signal that associated services (e.g., API-AG and API-AP) are ready to run.
- A periodic loop is established to generate new backups at regular intervals (every hour). Between backup operations, the script re-checks the database readiness and sends a notification to reload the index mapping in API-AG.

Overall, this API container script serves as a component for system stability and data integrity. It ensures that the database is always backed up and that restoration is handled automatically if needed. In addition, by notifying related services of the system's readiness, it facilitates smooth orchestration in a microservices architecture at startup. It might be less useful in Kubernetes deployment as backup system is proposed by Google than for the docker compose deployment. For a long term deployment, I would like to use an raspberry pi to host at home the web site and limiting fees. This way to manage backup was thought in this way.

* DATABASE Container Script for MySQL Database Initialization and Trigger Management

This container is based on the official MySQL 8.0 image and is configured to initialize the **Gabo_base** database with its necessary schema and triggers. It automatically sets up the database on container startup and ensures that the fundamental tables and triggers for handling player and game session data are in place. Even if it will be overwritten by the restore, it was required to have the correct schema.

The container Dockerfile and initialization script perform the following tasks:

- **Environment Configuration:**
 - The environment variable `MYSQL_DATABASE` is set to `Gabo_base`, which designates the target database for initialization.
- **SQL Initialization:**
 - The `init.sql` file is copied into the `/docker-entrypoint-initdb.d/` directory. This file contains the SQL commands to set up the database schema and triggers.
 - The initialization script includes commands to drop the `Gabo_base` database if it exists, create a new one, and switch to it. Without Drop table, I didn't manage load properly the mysqldump.
- **Table Creation:**
 - Two tables are defined:
 - **Joueurs:** Stores player data including an identifier, name, statistics such as number of games, total score, average score ratio, ranking ratio, and an Elo rating initialized to 500.
 - **Partie:** Records details of each game session, such as the game ID, number of players, individual player ID, score, rank, Elo variation for the session, and a timestamp. This table enforces a composite primary key on the game and player IDs and maintains a foreign key relationship with **Joueurs** to ensure referential integrity.
- **Trigger Implementation:**
 - An `after_insert_partie` trigger is created to automatically update player statistics after a new game session is recorded. This trigger increments the number of games played, updates the cumulative score, recalculates score and ranking ratios, and adjusts the Elo rating.
 - A complementary `after_delete_partie` trigger is defined to reverse these statistics when a game session is deleted, ensuring that the stored player data remains consistent.
 - Both triggers include error handling using a SQL state exit handler to signal any exceptions during trigger execution.

Overall, this container ensures that the `Gabo_base` MySQL database is freshly created with the correct schema, and it maintains data integrity through automatic trigger-based updates to player statistics.

* **NGINX Container for Proxying and Load-Balancing with NGINX**

This container is based on the official `nginx:stable-alpine` image and is configured to serve as a reverse proxy for the various application components. By using NGINX, the architecture gains a robust and efficient front-end proxy layer that routes incoming client requests to the appropriate backend services. This layer provides several benefits including improved security, load balancing, caching, and centralized access control.

The Dockerfile and associated NGINX configuration perform the following tasks:

- **Custom NGINX Configuration:**
 - The Dockerfile copies a custom `nginx.conf` file into the container (located at `/etc/nginx/nginx.conf`). This file defines the proxy rules and other configurations required for the setup.
 - The configuration specifies `worker_processes auto` along with `worker_connections 1024` to enable efficient handling of a high number of concurrent connections.
- **Upstream Definitions:**
 - Multiple upstream blocks are defined to correspond with each backend service. For instance, `flask_app` is mapped to `app-service:8000`, while `api_add_player`, `api_add_game`, and `api_rules` are mapped to their respective services and ports.
 - These upstream definitions allow NGINX to perform load balancing and distribute incoming requests efficiently among backend instances.
- **Server Block Configuration:**
 - A server block is defined to listen on port 80. Within this block, different URL locations are proxied to the corresponding upstream services:
 - Requests to the root (`/`) are forwarded to `flask_app`.
 - Paths starting with `/api-ap/` are directed to the player management service at `api_add_player`.
 - Requests to `/api-ag/` are routed to the game session service at `api_add_game`.
 - Calls to `/api-rules/` are sent to the rules service at `api_rules`.
- **Reason for Using NGINX:**

- In addition to acting as a reverse proxy, NGINX avoid CORS issues while deploying to remote vm.
- It encapsulates feedback such as error handling and circuit-breaker functionality to shield the backend services from direct exposure to the public internet.
- The loadbalancer included was suiting my idea to make multi-threaded APIS, overall it's not a bad thing to have.

Indeed, Nginx on port 80 will serve the entry point for http requests.

– Docker Compose Configuration for the Gabo Application

This `docker-compose.yml` file orchestrates the various containers that compose the full Gabo application. It uses version "3.9" of the Compose specification to define the services, networks, and volumes needed, ensuring that each component is correctly built, connected, and started in proper sequence. Below is a detailed breakdown of its components:

* **app Service:**

- **Build:** Built from the `./APP` directory using the specified `Dockerfile`.
- **Volumes:** Mounts the static files and templates from `./APP/Front_End` as well as shared utility code from `./utils`.
- **Expose:** Port 8000 is exposed within the Docker network.
- **Networks:** Connected to the shared `app_network` for inter-service communication.

* **api-add_player Service:**

- **Build:** Constructed from the `./API-AP` directory with a dedicated `Dockerfile`.
- **Environment:** Uses variables set in the `.env` file for proper configuration.
- **Volumes:** Maps the source code from `./API-AP` and shared utilities from `./utils`.
- **Expose:** Port 8010 is exposed.
- **Depends on:** The `data_base` service to ensure database availability.
- **Networks:** Connected via `app_network`.

* **api-add_game Service:**

- **Build:** Constructed from the `./API-AG` directory.
- **Environment:** Configured using the `.env` file.

- **Volumes:** Integrates the source code from `./API-AG` and shared utilities from `./utils`.
 - **Expose:** Port 8020 is exposed.
 - **Depends on:** The `data_base` service, ensuring that game-related data is stored properly.
 - **Networks:** Uses `app_network`.
- * **api-rules Service:**
- **Build:** Created from the `./API-RULES` directory.
 - **Volumes:** Maps the API code, shared utilities, and a read-only Kaggle configuration file (`kaggle.json`) to the container.
 - **Ports:** Exposes port 8030 to the host.
 - **Networks:** Communicates via `app_network`.
- * **data_base Service:**
- **Build:** Built from the `./Data_Base` directory.
 - **Container Name:** Named `Data_Base_Gabo`.
 - **Environment:** Configured with variables such as `DB_USER`, `DB_PASSWORD`, and `MYSQL_ROOT_PASSWORD` from the environment.
 - **Ports:** Maps MySQL's default port 3306.
 - **Volumes:** Mounts the initialization SQL script from `./Data_Base/init.sql` to automatically set up the database schema.
 - **Tmpfs:** Uses a temporary filesystem for `/var/lib/mysql` to improve performance.
 - **Healthcheck:** Configured to periodically ping the MySQL server to ensure it is operational.
 - **Networks:** Connected to the shared `app_network`.
- * **nginx Service:**
- **Build:** Built from the `./nginx` directory using its respective `Dockerfile`.
 - **Ports:** Binds host port 80 to the container, serving as the entry point for HTTP requests.
 - **Networks:** Uses the `app_network` for routing internal requests.
 - **Depends on:** The `app`, `api-add_player`, and `api-add_game` services, ensuring they are up before handling traffic.
- * **backup Service:**

- **Build:** Built from the `./Backup` directory.
- **Environment:** Uses variables from the `.env` file.
- **Volumes:** Mounts the backup storage directory (`./Backup/back_up`) and shared utilities (`./utils`).
- **Ports:** Exposes port 8080.
- **Depends on:** The `data_base` service, ensuring that backups are taken from an active database.
- **Networks:** Connected via `app_network`.

* **Networks:**

- A single network called `app_network` is defined using the bridge driver. This network facilitates secure and efficient communication among all the containers.

In summary, this Docker Compose configuration encapsulates the entire Gabo application stack, coordinating web, API, database, proxy, and backup services under a unified network.

– **Kubernetes Deployments for the Gabo Application**

These Kubernetes deployment configuration files define how the various services of the Gabo application are deployed into the cluster. Each deployment specifies the container image, replica count, resource requests and limits, environment variables, and exposed ports. The deployments work in concert to ensure that the individual microservices (such as API, web, and backup components) are scalable, secure, and can recover from failures. Below is a detailed breakdown of the deployments:

* **api-add-game Deployment:**

- **Container Details:** Uses the image `lagadglav/gabo-api-add_game:latest` and runs a single replica. The container listens on port 8020.
- **Environment Configuration:** Environment variables are set to configure the application's runtime context, including database connection details (host, database, user, and password) and various service URLs. Sensitive values such as `DB_USER` and `DB_PASSWORD` are sourced securely from Kubernetes secrets via `secretKeyRef`.
- **Resource Management:** It requests 256Mi of memory and 250m of CPU while imposing limits of 512Mi and 500m to adhere to resource constraints, especially in an autopilot cluster.

* **api-add-player Deployment:**

- **Container Details:** Deployed using the image `lagadglav/gabo-api-add_player:latest` with one replica. The container exposes port 8010.
 - **Environment Configuration:** Similar to the `api-add-game` service, it uses environment variables (with sensitive credentials from secrets) to interface with the database and other services. This ensures that the API for adding players operates with the correct configuration.
 - **Resource Management:** Resource requests and limits are identical to the `api-add-game` deployment.
- * **api-rules Deployment:**
- **Container Details:** This deployment uses the image `lagadglav/gabo-api-rules:latest` and runs with a single replica. The container is designed to listen on port 8030.
 - **Environment Configuration:** It is provided with environment variables for database details and service endpoints, ensuring that the rules service fetches and updates data appropriately.
 - **Resource Management:** This deployment requests lower resources (128Mi of memory and 100m of CPU) with limits set to 256Mi and 250m, reflecting its lighter workload compared to the other API services.
- * **app Deployment:**
- **Container Details:** The web application is defined by the image `lagadglav/gabo-app:latest` and exposes port 8000 on a single replica.
 - **Environment Configuration:** It receives similar service readiness, ensuring that the application can connect properly to its backend services.
 - **Resource Management:** The deployment requests 256Mi of memory and 250m of CPU, with limits of 512Mi and 500m.
- * **backup Deployment:**
- **Container Details:** Uses the image `lagadglav/gabo-backup:latest` to handle periodic database backups. It runs a single replica, with the container listening on port 8080.
 - **Environment Configuration:** Like other services, it is set up with environment variables for database connection, service URLs, and the backup directory path. Credentials are injected securely via Kubernetes secrets.

- **Resource Management and Volumes:** It requests 128Mi of memory and 100m of CPU (limits set at 256Mi and 250m). In addition, the backup deployment mounts a volume (in this case, an emptyDir volume) at `/Backup/back_up` to store backup files.
- * **data-base Deployment:**
- **Container Details:**
 - The `data-base` deployment uses the image `lagadglav/gabo-database:latest` and runs a single replica.
 - It exposes port 3306, which is the standard port for MySQL connections.
 - **Environment Configuration:**
 - Environment variables such as `DB_HOST`, `DB_DATABASE`, `DB_USER`, and `DB_PASSWORD` are set to properly configure the database connection.
 - Sensitive credentials like `DB_USER` and `DB_PASSWORD` (as well as `MYSQL_ROOT_PASSWORD`) are injected via Kubernetes secrets, ensuring security.
 - **Resource Management:**
 - The deployment requests 256Mi of memory and 250m of CPU, with limits set to 512Mi and 500m respectively, to ensure efficient resource usage.
 - **Persistent Storage:**
 - A volume mount is configured for `/var/lib/mysql` using a PersistentVolumeClaim named `mysql-pvc`, which ensures that the database data is persisted across pod restarts.
- * **PersistentVolumeClaim for MySQL:**
- A PersistentVolumeClaim (PVC) named `mysql-pvc` is defined with access mode `ReadWriteOnce` and a storage request of 5Gi.
 - This PVC guarantees that the MySQL database has stable and persistent storage, which is critical for data continuity and recovery.
- * **nginx Deployment:**
- **Container Details:**
 - The `nginx` service is deployed using the image `lagadglav/gabo-nginx:latest` with one replica.
 - It exposes port 80, making it the entry point for all HTTP traffic.

- **Resource Management:**
- The deployment requests 256Mi of memory and 250m of CPU, with resource limits of 512Mi and 500m respectively, ensuring the service has sufficient resources while keeping resource usage under control.
- **Role in the Architecture:**
- The nginx deployment acts as a reverse proxy, routing external traffic to the appropriate internal services (such as the web application and API services), thereby centralizing access control and simplifying domain management.

– GitHub Actions Deployment Workflows

The following GitHub Actions workflows automate the deployment process for the Gabo application. One workflow deploys the repository directly to a virtual machine (VM) using Docker Compose, while the other workflow builds and deploys Docker images to a Google Kubernetes Engine (GKE) Autopilot cluster. Both workflows ensure that the latest source code is fetched, built, and deployed consistently, but they target different environments to meet varying operational requirements.

Deploy Repository to VM:

This workflow is triggered on a push to the `production-docker` branch and performs the following steps:

* **Checkout Repository:**

- Uses `actions/checkout@v3` to clone the repository, ensuring that the latest code is readily available.

* **Clean Target Directory on VM:**

- Executes an SSH command via `appleboy/ssh-action@master` to remove the existing contents from the target directory on the VM. This ensures a clean slate for the new deployment.

* **Copy Repository to VM:**

- Uses `appleboy/scp-action@master` to securely copy the entire repository to the VM's application directory.

* **Create .env on VM:**

- Runs an SSH command to create a `.env` file in the target directory. The environment variables for the application are written to this file using data stored in GitHub Secrets.

* **Deploy Docker Compose on VM:**

- Executes commands via SSH to navigate to the application directory, bring down any running Docker Compose services, rebuild the containers, and then restart the application stack in detached mode. This sequence automates the rebuilding and redeployment of the application.

Deploy to GKE Autopilot:

This workflow is triggered on a push to the `production` branch and automates the deployment to a GKE Autopilot cluster with the following steps:

*** Checkout Code:**

- Uses `actions/checkout@v2` to obtain the latest code from the repository.

*** Set Up Google Cloud SDK:**

- Uses the `google-github-actions/setup-gcloud@v0.2.0` action with a service account key (stored in GitHub Secrets) and the GCP project ID, thereby preparing the environment for GKE deployments.

*** Install GKE Auth Plugin and Update gcloud Components:**

- Runs shell commands to install the GKE gcloud authentication plugin and update the CLI, ensuring secure communication with the cluster.

*** Authenticate Docker to Docker Hub:**

- Authenticates Docker using GitHub Secrets so that built images can be pushed to Docker Hub.

*** Build and Push Docker Images:**

- Executes Docker build and push commands for all application components (web, API-add_game, API-add_player, nginx, database, backup, and API-rules) using their respective Dockerfiles. This step ensures that each service is packaged into the latest container image.

*** Pull MySQL Image:**

- Uses a Docker pull command to fetch the MySQL image from Docker Hub, ensuring the latest database image is available.

*** Configure kubeconfig for GKE Access:**

- Sets up the kubeconfig file by decoding a base64-encoded configuration stored in GitHub Secrets. It then obtains cluster credentials using `gcloud container clusters get-credentials` and sets the correct Kubernetes context.

*** Create Database Secret from Template:**

- Processes a secret template file to create a Kubernetes secret containing database credentials. This secret is applied to the cluster using `kubectl apply`.

*** Deploy Applications to GKE:**

- Runs a series of `kubectl apply` commands to deploy all Kubernetes manifests (persistent volumes, deployments, and services) for the MySQL, API, web, backup, and nginx components to the GKE cluster. This step ensures that the entire application stack is deployed and updated in the cluster.

In summary, both workflows streamline the deployment process by automating essential tasks such as code checkout, artifact building, secure configuration, and deployment. While the VM deployment directly manages the Docker Compose stack on a virtual machine, the GKE Autopilot workflow builds container images and deploys them to a managed Kubernetes cluster.

3 Deployment and Usage Guide

3.1 Docker Compose local deployment

To run locally the web application thanks to Docker-Compose :

- git clone the Gabo_Dev/production-docker
- copy the following content into `.env` along Docker-Compose :
DB_HOST=data_base
DB_PORT=3306
DB_USER=root
DB_PASSWORD=Gabo
DB_DATABASE=Gabo_base
BACKUP_PATH=/Backup/back_up
FLASK_APP_URL=http://app:8000/ready
API_AP_URL=http://api-add_player:8010/ready
API_AG_URL=http://api-add_game:8020/ready
API_AG_URL_INDEX=http://api-add_game:8020/reload_indexbyname
- run: `sudo docker-compose up --build`
- Access on you web browser at: `http://localhost/`

3.2 Docker Compose deployment Access

To access the web application deployment thanks to Docker-Compose :
`http://34.10.87.199/` Here is the link that we are using to enter real games.

Feel free to take a look but please prefer doing your test on the kubernetes deployment.

3.3 Kubernetes deployment Access

You should be able to reach the web application with this address : <http://35.226.163.123/> Here you can do whatever you want to try to break the web app. I already enter some of my fellow testers to the database, you can add new players and new games to see what it produces.

3.4 Usage Guide

Once on the web, you can reach the four pages. On add player page, you can add a player by inserting an id and the name desired. On add game page, you can create a new game and then adding players to the table by their ids or names. On scoreboard page, you can access the full scoreboard, if you click on the card you can get additional information. Still on this page, you can

4 GitHub Repository and DockerHub Images

GitHub Repository: Gabo_Dev

DockerHub Images:

- App Image
- API-Add_Game Image
- API-Add_Player Image
- API-Rules Image
- NGINX Image
- Database Image
- Backup Image

5 Conclusion

To be honest, it is the first time I'm building a that large project and it is scheduled to enhance with new features like display recap of games, adding each round in the game table with the value of each card avoiding each player to do his/her own counting. Moreover the front end design needs to be done, it will be probably be the next step.

I tested some new architectures and logic ways such as the index map with patches from an API to an other, some kind of threading and the backup container without perfectly knowing if it is the best way to it.